

Análise Assintótica de Algoritmos **Recursivos**

Algoritmos de decremento e conquista

Algoritmos recursivos de decremento e conquista

Decremento e conquista

- 1 Remova b elementos da entrada (tamanho n).
- 2 Resolva recursivamente o problema com a entrada restante (tamanho $n - b$), demandando ' a ' chamadas recursivas.
- 3 Combine a solução do subproblema com os b elementos removidos para formar a solução do problema original.

Assim, o total $T(n)$ de instruções executadas é descrito pela recorrência:

$$T(n) = a \cdot T(n - b) + f(n)$$

onde:

- a é o número de chamadas recursivas, $n - b$ é o tamanho da entrada em cada chamada,
- $f(n)$ é o tempo gasto na remoção de elementos e inclusão dos elementos removidos.

Exemplo: Selection Sort por decremento e conquista

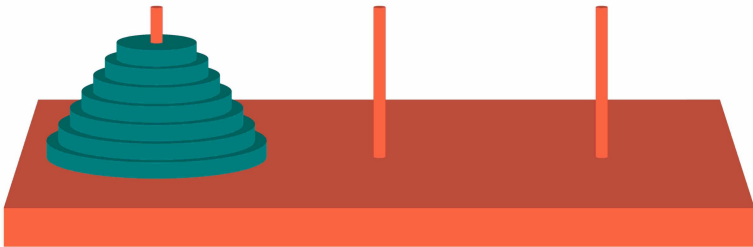
```
1 def selection_sort(A, i, n):  
2     if i < n - 1:  
3         # Encontra o índice do menor elemento no subvetor A[i..n-1]  
4         min_idx = i  
5         for j in range(i + 1, n):  
6             if A[j] < A[min_idx]:  
7                 min_idx = j  
8         # Troca o menor elemento com A[i]  
9         A[i], A[min_idx] = A[min_idx], A[i]  
10        # Chamada recursiva para o restante do vetor  
11        selection_sort(A, i + 1, n)
```

Desejamos saber a complexidade de `selection_sort(A, 0, n)` (usando $i = 0$, vetor inteiro).

- Busca do menor elemento e troca: $f(n) \in \Theta(n)$ operações.
- Chamada recursiva ($a = 1, b = 1$): resolve o subproblema de tamanho $n - 1$.

$$T(n) = T(n - 1) + \Theta(n)$$

Exemplo: Torres de Hanói por decremento e conquista



Exemplo: Torres de Hanói por decremento e conquista

Mover n discos do pino *origem* para o pino *destino* usando um pino *auxiliar*:

```
1 def hanoi(n, origem, destino, auxiliar):  
2     if n > 0:  
3         # Move n - 1 discos para o pino auxiliar  
4         hanoi(n - 1, origem, auxiliar, destino)  
5         # Move o disco restante (maior) para o destino  
6         print(f"Mover disco {n} de {origem} para {destino}")  
7         # Move os n - 1 discos do auxiliar para o destino  
8         hanoi(n - 1, auxiliar, destino, origem)
```

Para mover n discos:

- Movimento do disco maior e impressão ($f(n)$): $\Theta(1)$ operações.
- Chamadas recursivas ($a = 2, b = 1$): duas resoluções do subproblema de tamanho $n - 1$.

$$T(n) = 2 \cdot T(n - 1) + \Theta(1)$$

Simplificação Θ de recorrências de decremento e conquista

Se $T(n) = a \cdot T(n - b) + f(n)$, com $f(n) = \Theta(n^d \log^e n)$,

a	Solução
> 1	$\Theta(a^{n/b})$
$= 1$	$\Theta(n \cdot f(n))$

Exemplos:

- 1 Torres de Hanói: $T(n) = 2T(n - 1) + \Theta(1)$.
- 2 Ordenação por seleção: $T(n) = T(n - 1) + \Theta(n)$.
- 3 $T(n) = 8T(n - 3) + \textcolor{red}{f(n)}$.
- 4 $T(n) = T(n - 3) + n \log n$.

Simplificação Θ de recorrências de decremento e conquista

Se $T(n) = a \cdot T(n - b) + f(n)$, com $f(n) = \Theta(n^d \log^e n)$,

a	Solução
> 1	$\Theta(a^{n/b})$
$= 1$	$\Theta(n \cdot f(n))$

Exemplos:

- 1 Torres de Hanói: $T(n) = 2T(n - 1) + \Theta(1) \in \Theta(2^n)$.
- 2 Ordenação por seleção: $T(n) = T(n - 1) + \Theta(n) \in \Theta(n^2)$.
- 3 $T(n) = 8T(n - 3) + f(n) \in \Theta(8^{n/3}) = \Theta(2^n)$.
- 4 $T(n) = T(n - 3) + n \log n \in \Theta(n \cdot (n \log n)) = \Theta(n^2 \log n)$.

Exercícios

Exercício: Solução de Recorrências (Θ) – Decremento e Conquista

Para cada recorrência da forma $T(n) = a \cdot T(n - b) + f(n)$, com $f(n) = \Theta(n^d \log^e n)$, determine a sua solução em notação Θ :

❶ $T(n) = T(n - 1) + 1$

❷ $T(n) = 2T(n - 1) + 1$

❸ $T(n) = T(n - 1) + n$

❹ $T(n) = 3T(n - 2) + n^2$

❺ $T(n) = T(n - 2) + n^2$

❻ $T(n) = 2T(n - 3) + n \log n$

❼ $T(n) = T(n - 1) + n \log^2 n$

❽ $T(n) = 4T(n - 1) + 2^n$

Gabarito: Solução de Recorrências (Θ)

① $T(n) \in \Theta(n)$

$$a = 1 \implies \Theta(n \cdot f(n)) = \Theta(n \cdot 1).$$

② $T(n) \in \Theta(2^n)$

$$a = 2 > 1, b = 1 \implies \Theta(a^{n/b}) = \Theta(2^n).$$

③ $T(n) \in \Theta(n^2)$

$$a = 1 \implies \Theta(n \cdot f(n)) = \Theta(n \cdot n).$$

④ $T(n) \in \Theta(3^{n/2})$

$$a = 3 > 1, b = 2 \implies \Theta(a^{n/b}) = \Theta(3^{n/2}).$$

⑤ $T(n) \in \Theta(n^3)$

$$a = 1 \implies \Theta(n \cdot f(n)) = \Theta(n \cdot n^2).$$

⑥ $T(n) \in \Theta(2^{n/3})$

$$a = 2 > 1, b = 3 \implies \Theta(a^{n/b}) = \Theta(2^{n/3}).$$

⑦ $T(n) \in \Theta(n^2 \log^2 n)$

$$a = 1 \implies \Theta(n \cdot f(n)) = \Theta(n \cdot n \log^2 n).$$

⑧ $T(n) \in \Theta(4^n)$

$$a = 4 > 1, b = 1 \implies \Theta(a^{n/b}) = \Theta(4^n).$$

Exercício: Recorrência de Decremento e Conquista

Considere o algoritmo abaixo que realiza a busca de um elemento x em uma lista desordenada A de tamanho n , projetado segundo a estratégia de **decremento e conquista**:

```
1 def busca_linear_rec(A, n, x):  
2     if n == 0:  
3         return False  
4     if A[n - 1] == x:  
5         return True  
6     return busca_linear_rec(A, n - 1, x)
```

Questão:

- 1 Identifique os parâmetros a , b e $f(n)$ no modelo de decremento e conquista $T(n) = a \cdot T(n - b) + f(n)$ (pior caso).
- 2 Resolva a recorrência e determine a complexidade assintótica em notação Θ . Tem complexidade pior que uma busca local iterativa?

Gabarito: Modelagem e Resolução da Recorrência

1. Modelagem da Recorrência (Pior Caso)

O pior caso ocorre quando x não está presente no vetor ou é o primeiro elemento ($A[0]$):

- **Remoção de elementos** ($b = 1$): A cada chamada, o tamanho do problema é reduzido em 1 elemento ($n - 1$).
- **Subproblema recursivo** ($a = 1$): Executa apenas 1 chamada recursiva.
- **Custo local** ($f(n)$): Verificações condicionais e indexação demandam tempo constante: $f(n) = \Theta(1)$.

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 0 \\ T(n-1) + \Theta(1), & \text{se } n > 0 \end{cases}$$

2. Resolução da Recorrência em Notação Θ

Como $a = 1$, $b = 1$, e $f(n) = \Theta(1)$, temos que $T(n) \in \Theta(n \cdot 1) = \Theta(n)$.

Ou seja, tem a mesma complexidade assintótica de uma busca linear iterativa, que é $\Theta(n)$.